



Instructor	Mr. Shehryar Raza
Lab Instructor	Muzamil Ali
Semester	Fall 2024

Department of Computer Science ITU, Lahore
Pakistan

Instructions

Please submit a single cpp file titled as **"Bscs23xxx_Lab03.cpp"**

Iterators

Part A

Task 01: Library Book Arrangement

Problem Statement: (Do this using STL Iterator)

You are a librarian, and you need to arrange a collection of books on a shelf. The books are represented by their heights in a list. You want to arrange the books such that they form the

longest sequence of books where each subsequent book is taller than the previous one. Your task is to implement a function to find the length of the longest sequence of increasing heights.

Example 01:

```
Input: heights = [5, 3, 4, 8, 6, 7]
Output: 4
Explanation:
The longest sequence of increasing heights is [3, 4, 6, 7].
```

Example 02:

```
Input: heights = [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
Explanation:
The longest sequence of increasing heights is [2, 3, 7, 101].
```

Part B

Problem Statement:

You are required to design and implement a **DynamicArray** class in C++ that can store elements of any type. Additionally, you must create a custom **Iterator class** for this DynamicArray, enabling users to traverse through the elements of the array using standard iterator operations.

Part 1: Dynamic Array Implementation

You have already done it in your previous Lab

Part 2: Iterator Implementation

Iterator Class:

- Implement a nested class Iterator within DynamicArray. The Iterator class should provide the following functionalities:
 - Data Members:
 1. T* ptr: A pointer to the current element in the array that the iterator is pointing to.
 - Member Functions:
 1. Constructor:
 - Iterator(T* p);

- Initializes the iterator with a pointer to an element in the array.
- 2. Dereference Operator (*):
 - T& operator*() const;
 - Returns a reference to the element at the current iterator position.
- 3. Arrow Operator (->):
 - T* operator->() const;
 - Returns a pointer to the element at the current iterator position.
- 4. Pre-Increment Operator (++it):
 - Iterator& operator++();
 - Advances the iterator to the next element and returns a reference to the updated iterator.
- 5. Post-Increment Operator (it++):
 - Iterator operator++(int);
 - Advances the iterator to the next element and returns the iterator before it was incremented.
- 6. Equality Comparison (==):
 - bool operator==(const Iterator& other) const;
 - Returns true if both iterators point to the same element.
- 7. Inequality Comparison (!=):
 - bool operator!=(const Iterator& other) const;
 - Returns true if the iterators point to different elements.